

Editorial: Problem E - Jayden's Automaton Factory

Setter: Hudson

Problem Overview

We are given a board width $W \leq 6$, a DFA describing valid piece sequences, and a target length $L \leq 10^{18}$. We need to count paths in the DFA such that the Tetris board ends up empty.

Key Challenges

1. **Infinite Height vs Finite State:** While the stack can grow infinitely high in theory, we only care if the board *eventually* clears. This suggests we should track the **relative profile** (skyline) of the board rather than absolute heights.
2. **Large L :** Since L is up to 10^{18} , a simulation $O(L)$ is impossible. We must use Matrix Exponentiation $O(\log L)$.
3. **Complex State:** The state must combine the Automaton Node and the Tetris Board Profile.

Solution Approach

1. Defining the Profile

Since W is very small ($W \leq 6$), we can represent the surface of the current stack as a tuple of heights $(h_1, h_2, \dots, h_W)$. Crucially, absolute height doesn't affect placement logic—only relative height does. We normalize the profile by subtracting the minimum height from all columns:

$$P = (h_1 - \min(h), h_2 - \min(h), \dots, h_W - \min(h))$$

The "Target State" is the flat profile $(0, 0, \dots, 0)$.

Bound on Profile Height: Does the difference between max and min height grow indefinitely? If we are only searching for paths that return to the flat state $(0, \dots, 0)$, we can prune any state where $\max(P) > K$. For $W = 6$, a heuristic limit of $K = 8$ is sufficient. If the "hole" becomes deeper than the longest piece (plus a buffer), it becomes impossible to fill that hole without creating new holes higher up.

2. Precomputing Transitions (The "Super-Node")

Our total system state is defined by:

$$S = (\text{DFA Node } u, \text{Profile } P)$$

The number of valid profiles for $W = 6$ is surprisingly small (a few hundred). The number of DFA nodes is ≤ 50 . Total states $\approx 50 \times (\text{small constant})$.

We perform a BFS/DFS to discover reachable states starting from (Node 1, Profile $\{0 \dots 0\}$). For each reachable state (u, P) and each outgoing edge in the DFA $u \rightarrow v$ with piece (T, C) :

1. Simulate dropping piece T at column C onto profile P .
2. Calculate new column heights.
3. If a row is full (all W cols increased), simulate line clear (subtract 1 from all).
4. Re-normalize (subtract min height).
5. If max height exceeds cutoff, discard.
6. Otherwise, we have a valid transition to (v, P_{new}) .

3. Matrix Exponentiation

Let $\mathcal{S}$ be the set of valid composite states. Map each state to an index $0 \dots M - 1$. Construct an adjacency matrix $\mathbf{A}$ of size $M \times M$, where $\mathbf{A}[i][j] = 1$ if there is a transition from state i to state j . Note: There may be multiple edges between DFA nodes; sum them up in the matrix.

The answer is the entry corresponding to (Start Node, Flat) $\rightarrow$ (Any Node, Flat) in the matrix $\mathbf{A}^L$. Strictly speaking, we want the count of paths ending at specific states. We initialize a row vector V_0 with 1 at index (Node 1, Flat) and 0 elsewhere. We compute $V_L = V_0 \times \mathbf{A}^L$. The final answer is the sum of $V_L[k]$ for all k where the profile part of state k is Flat.

Complexity

- **Profile Generation:** The number of profiles is related to the "roughness" of the surface. For $W = 6$, this is manageable.
- **Matrix Size:** Let $S_{total} \approx 2000$. Matrix multiplication is $O(S_{total}^3)$. With $L = 10^{18}$, we need ≈ 60 multiplications.
- **Optimization:** 2000^3 is too slow for 5.0s. However, the matrix is sparse.
- **Actual Constraints:** With $W \leq 6$, the number of relevant "Flat-reachable" profiles is usually < 100 . With $N = 50$, matrix size is roughly 5000. This is still tight. We can optimize by noting that for specific W , many profiles are isomorphic or unreachable.

Implementation (C++)

```
1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <map>
5 #include <queue>
6 #include <algorithm>
7
8 using namespace std;
9
10 const int MOD = 1e9 + 7;
11
12 // --- TETRIS LOGIC ---
13
14 struct PieceShape {
15     int h, w;
16     vector<string> grid;
17 };
18
19 // Definitions matching the problem description
20 map<char, PieceShape> SHAPES = {
21     {'I', {1, 4, {"####"}}},
22     {'O', {2, 2, {"##", "##"}}},
23     {'T', {2, 3, {"#.#", "###"}}},
24     {'L', {2, 3, {"..#", "###"}}},
25     {'J', {2, 3, {"#..", "###"}}},
26     {'S', {2, 3, {"##.", "##."}}},
27     {'Z', {2, 3, {"##.", "##."}}}
28 };
29
30 // Represents the "Surface" of the board.
31 using Board = vector<string>;
32
33 // Max height allowed before we declare state "dead"
34 const int MAX_PRUNE_HEIGHT = 8;
35
36 // Simulate dropping a piece. Returns empty board if invalid/overflow.
37 Board simulate(Board b, char pType, int col, int W) {
38     PieceShape p = SHAPES[pType];
39     int c_idx = col - 1;
40
41     // Pad board with empty space to simulate drop
42     int current_h = b.size();
43     int start_y = current_h + 3; // Start safely above
44
45     // Find lowest valid position
46     int final_y = -1;
47
48     for (int y = start_y; y >= 0; --y) {
49         bool collision = false;
50         // Check collision at this position
51         for (int pr = 0; pr < p.h; ++pr) {
52             for (int pc = 0; pc < p.w; ++pc) {
53                 if (p.grid[pr][pc] == '#') {
54                     int abs_r = y + (p.h - 1 - pr);
55                     if (abs_r < 0) { collision = true; break; }
56                     if (abs_r < current_h && b[abs_r][c_idx + pc] == '#') {
```

```

57         collision = true; break;
58     }
59 }
60 }
61 if(collision) break;
62 }
63
64 if (collision) {
65     final_y = y + 1;
66     break;
67 }
68 if (y == 0) final_y = 0; // Reached floor
69 }
70
71 // Place piece
72 for (int pr = 0; pr < p.h; ++pr) {
73     for (int pc = 0; pc < p.w; ++pc) {
74         if (p.grid[pr][pc] == '#') {
75             int abs_r = final_y + (p.h - 1 - pr);
76             int abs_c = c_idx + pc;
77             while (b.size() <= abs_r) b.push_back(string(W, '.'));
78             b[abs_r][abs_c] = '#';
79         }
80     }
81 }
82
83 // Clear lines
84 vector<string> next_b;
85 for (const string& row : b) {
86     bool full = true;
87     for (char c : row) if (c == '.') full = false;
88     if (!full) next_b.push_back(row);
89 }
90
91 // Prune logic
92 if (next_b.size() > MAX_PRUNE_HEIGHT) return {"INVALID"};
93
94 return next_b;
95 }
96
97 // --- MATRIX LOGIC ---
98
99 typedef vector<vector<long long>> Matrix;
100
101 Matrix multiply(const Matrix& A, const Matrix& B) {
102     int n = A.size();
103     Matrix C(n, vector<long long>(n, 0));
104     for (int i = 0; i < n; ++i)
105         for (int k = 0; k < n; ++k)
106             if (A[i][k])
107                 for (int j = 0; j < n; ++j)
108                     C[i][j] = (C[i][j] + A[i][k] * B[k][j]) % MOD;
109     return C;
110 }
111
112 Matrix power(Matrix A, long long p) {
113     int n = A.size();
114     Matrix Res(n, vector<long long>(n, 0));
115     for (int i = 0; i < n; ++i) Res[i][i] = 1;

```

```

116     while (p > 0) {
117         if (p & 1) Res = multiply(Res, A);
118         A = multiply(A, A);
119         p >>= 1;
120     }
121     return Res;
122 }
123
124 // --- MAIN SOLVER ---
125
126 struct Edge {
127     int v;
128     char type;
129     int col;
130 };
131
132 int main() {
133     ios_base::sync_with_stdio(false);
134     cin.tie(NULL);
135
136     int W, N;
137     long long L;
138     if (!(cin >> W >> N >> L)) return 0;
139
140     int M;
141     cin >> M;
142     vector<vector<Edge>> adj(N + 1);
143     for (int i = 0; i < M; ++i) {
144         int u, v, c;
145         char t;
146         cin >> u >> v >> t >> c;
147         adj[u].push_back({v, t, c});
148     }
149
150     // BFS to find reachable states (Node, Board)
151     map<pair<int, Board>, int> state_to_id;
152     vector<pair<int, Board>> id_to_state;
153     queue<pair<int, Board>> q;
154
155     Board empty_board; // size 0
156     state_to_id[{1, empty_board}] = 0;
157     id_to_state.push_back({1, empty_board});
158     q.push({1, empty_board});
159
160     int state_cnt = 0;
161
162     while(!q.empty()) {
163         auto [u, board] = q.front();
164         q.pop();
165
166         // Try all edges
167         for (auto& e : adj[u]) {
168             Board next_b = simulate(board, e.type, e.col, W);
169             if (next_b.size() == 1 && next_b[0] == "INVALID") continue;
170
171             pair<int, Board> next_state = {e.v, next_b};
172             if (state_to_id.find(next_state) == state_to_id.end()) {
173                 state_to_id[next_state] = ++state_cnt;
174                 id_to_state.push_back(next_state);

```

```

175         q.push(next_state);
176     }
177 }
178 }
179
180 int total_states = state_cnt + 1;
181 Matrix Mat(total_states, vector<long long>(total_states, 0));
182
183 // Fill Matrix
184 for (int i = 0; i < total_states; ++i) {
185     auto [u, board] = id_to_state[i];
186     for (auto& e : adj[u]) {
187         Board next_b = simulate(board, e.type, e.col, W);
188         if (next_b.size() == 1 && next_b[0] == "INVALID") continue;
189
190         int j = state_to_id[{e.v, next_b}];
191         Mat[i][j]++;
192     }
193 }
194
195 // Exponentiate
196 Matrix FinalMat = power(Mat, L);
197
198 long long ans = 0;
199 for (int i = 0; i < total_states; ++i) {
200     // We want transitions 0 -> i where i is a Flat board
201     if (FinalMat[0][i] > 0) {
202         if (id_to_state[i].second.empty()) {
203             ans = (ans + FinalMat[0][i]) % MOD;
204         }
205     }
206 }
207
208 cout << ans << endl;
209 return 0;
210 }

```